Let's design production grade API's

1

⇒ npm init Y

⇒ npm i express

⇒ change to type : module in pakage.json

⇒ we are using "mongoDB" for now

⇒ Also install mongoose library.

We can directly ask express to talk to DB, but we tend to make it in such a way that express talks to mongoose and then mongoose talks to the db.

These are called ORM (Object-Relational Mapping) eg SQL and ODM (Object-Document Mapping) eg NoSQL, MongoDB

Basic File Structure

server.js

SRC ↳ app.js

↳ controller (handles business logic

routes (defines URLs / endpoints)

models (handles data, defines schema and rules)

utils : utilities eg dbconnect

mail

etc (helper fun)

/register
/login
/verify
/logout

firstname
lastName
password
email

Frameworks like nest'js, spring boot etc have their quite fixed folder structure.

A framework is like a skeleton + rules + tools for our project. Without a framework, we design everything from scratch.

⇒ .env or enviornment variable file should never be pushed on github. It contains secrets, keys, API keys, db URLS etc 2

⇒ .gitignore file to make sure you don't push .env or node-modules.

⇒ .env.example ⇒ This helps others to know what exact variables are required, without exposing .env.

⇒ We use .dotenv package: lets your app read variables from .env file, because Node.js cannot read it directly. dotenv loads those values to "process.env"

eg:

```
const PORT = process.env.PORT || 5000
```

⇒ import express from "express"

```
const app = express()

export default app
```

Note: Add NODE_ENV=development (tells which mode is app in)

⇒ ~~Connect db in server.js~~ Starting server

```
const start = async () => {
    app.listen(PORT, () => {
        console.log(`Port is running at ${PORT} in ${process.env.NODE_ENV}`)
    })
)

start().catch((err) => {
    console.error("failed to start server", err)
    process.exit(1)
})
```

modules folder will have user, cart so the files will be

user.model.js	cart.model.js
user.controller.js	cart.controller.js
user.service.js	cart.service.js
user.routes.js	cart.routes.js

3

The common folder has config, middleware
We are doing all this because we want "seperation of concern". Thus we try keeping common logic in the common folder.

Assignment : difference btw URL and URI.

Add MONGO_URI

Note : Remember, db connection can fail and db is always in different continent.

This note make sure that we use "async await" always.

⇒ console.log(`MongoDB connected : ${conn.connection.host}`);
⇓
prints a message when app is successfully connects to MongoDB

⇒ conn.connection.host gives us DB server address our app is connected to

⇒ We need to have a standard of response, standard of error.

⇒ We need a standard structure.

⇒ We will keep this structure in common folder Inside utils.

Note: During any standarization, classes are your best friend

eg Inside file: api-response.js

```
class ApiResponse {
    static ok (res, message, data = null) {
        return res.status(200).json({
            success: true,
            message,
            data
        })
    }
}
```

Static is used to define methods or properties that belong to the class itself, not to its instance.

→ code continued

```
    static created (res, message, data = null) {
        return res.status(201).json({
            success: true,
            message,
            data
        })
    }

    static noContent (res) {
        return res.status(204).send()
    }
```

Now our API response is standarized.

Now next we will standarize the error

⇒ api-error.js

```
class ApiError extends Error {
  constructor (statusCode, message) {
    super(message)
    this.statusCode = statusCode
    this.isOperational = true
    Error.captureStackTrace(this, this.constructor)
  }
}
```

Note. isOperational is a flag used in custom errors.

Error.captureStack is used to manually create a clean stack trace for an error.

Now we are creating static helper methods:

eg: ← factory method

```
static badRequest(message = "Bad Request") {
  return new ApiError(400, message)
}
}
```

Validation Library: like zod, yup, archtype, joi etc

Zod, archtype preferred if code is in TS

In JS prefer yup, express-validator, joi

DTO (Data Transfer Object) is a design pattern

This transfer our data into object for us to send to server

It is an object used to transfer data between layers of our app.

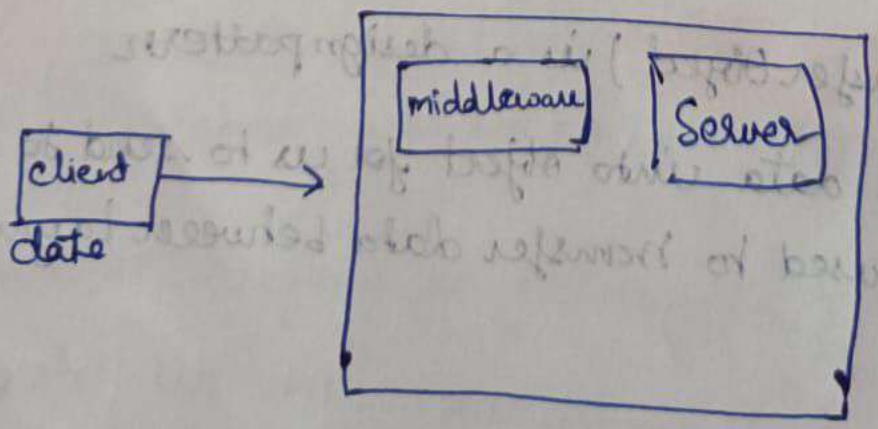
We need to make sure the shape of our data is maintained while transferring from one layer to another.

This shape / structure / schema.

DTO is a concept / design pattern. Zod, joi are used to implement this concept.

⇒ inside base-dto.js

```
import Joi from "joi";
class BaseDTO {
    static schema = Joi.object({})
    static validate(data) {
const{error, value} = this.schema.validate(data, {
            abortEarly : false,
            stripUnknown : true
        })
    if(error){
        const errors = error details.map((d) => d.message)
        return { errors, value: null }
    }
    return { errors : null, value }
}
}
```



⇒ middleware folder has file
⇒ validate.middleware.js

```
// import ApiError from ------;
const validate = (Dtoclass) ⇒ {
    return (req, res, next) ⇒ {
const {errors, value} = DtoClass.validate(req.body)
  if(errors){
      throw Api.badRequest(Errors.join(";"))
  }
  req.body = value
```

⇓
replaces original input with validated, sanitized data.

```
  next();
}
export default validate
```

⇒ Inside modules folder make folder auth and cart

↳ module
↳ auth
  ↳ dto
  ↳ auth.controller.js
  ↳ auth.middleware.js
  ↳ auth.model.js
  ↳ auth.routes.js
  ↳

Step 1: ~~DTO~~ Database Design for user

name, uniqueId
email,
password
role
isVerified

verificationToken
refreshToken
resetPasswordToken
resetPasswordExpires

createdAt
updatedAt

=>

auth.model.js

defines structure of our data

```
import mongoose from "mongoose";
const userSchema = new mongoose.Schema({
    name : {
        type : String,
        trim : true
        minLength : 2,
        maxLength : 50,
        required : [true, "Name is required"]
    },
    email : {
        . . . . . . . . .
export default mongoose.model("User", userSchema)
```

→ mongoose.model ("User", userSchema)
↳ this line registers and with help of mongoose, the schema is compiled into a Model class.

"User" is the name of the model, is created using mongoose.model()

Next step after Database design

Step 2 : DTO

```
import joi from "joi"
import BaseDto from .....
class RegisterDto extends BaseDto {
    static schema = Joi.object({
    name : Joi.string().trim().min(2).max(50).required()
    email : Joi.string().email().lowercase().required(),
    password : Joi.string()
                .pattern(.....)
                .min(8).required()
                .message("Password must contain 8 char")
```

helps to give custom msg

```
    role : Joi.string().valid("customer", "seller").
            .default("customer")

export default RegisterDto
```

⇒ Express5 has by default has async handles.

⇒ Use Dto class in our routes for validation.

routes file has ⇒

router post ("/register", validate (RegisterDto), controller.register

⇒ auth.service has the business logic

⇒ controller has the code, which service is being called. Its like a mapping file.

Inside auth.service

1. Gather data and validation (assume DTO has done it)

2. import User from "./auth/.model.js" ← using wrapper User

const register = async({name, email, password, role}) ⇒ {

await existing = await User.findOne({email})

if (existing) throw Api.Error.conflict ("Email already exist");

const {rawToken, hashedToken} = generateResetToken()

const user = await User.create({
name, email, password, role, verificationToken: hashToken

3)

npm i jsonwebtoken

token is just a long string, can be made using crypto, hashed, uuid etc.

const { rawToken , hashedToken } = generateResetToken()

⇓ will send to user

⇓ will store in DB

To delete stuff

```
const userObj = user.toObject()
delete userObj.password
delete userObj.verificationToken
return userObj;
```

→ To get the MONGO_URI, go to MongoDB atlas and create cluster
→ connect to your cluster and inside the drivers you'll get the string.. Also create a user and password and replace its contents in the URI. Also do "ADD IP Access list to 0.0.0.0/0.